



MYTHIC

YOLO Retraining Guide for Mythic AMP

May 2026

Document Version 1.0
SDK Version v26.05.0

1	Introduction	3
2	Hardware Requirements	3
3	Prerequisites	3
4	Ultralytics' yolo CLI.....	3
5	Task Configuration.....	4
5.1	Dataset Setup	5
5.1.1	Using Standard Datasets	5
5.1.2	Using a Custom Dataset.....	6
6	Model Configuration Files	7
6.1	Reducing number of classes in a model configuration file.....	7
7	Conversion and Retraining Steps.....	8
7.1	Generate a Trained Floating Point Model.....	8
7.2	Export the Trained Floating Point Model to ONNX	9
7.3	Convert the Floating Point model to Structural.....	9
7.4	Convert the Structural Model to Retraining	9
7.5	Retrain.....	10
7.5.1	Hyper-parameters	10
7.5.2	Retraining Parameters	11
7.6	Change Model Input Resolution	12
7.7	Optimize per-channel ADC ranges (optional).....	12
7.8	Convert to Evaluation Model	12
7.9	Compute Accuracy Metrics	13
7.9.1	M2000	13
7.9.2	M1076C	13
7.10	Generate Artifact.....	14
7.11	Firmware Compilation.....	14
7.12	Benchmark AMP Inference (M1076C only).....	16

MYTHIC

1 Introduction

Mythic[®] provides Object-detection, Segmentation, and Pose models from Ultralytics as part of its model-zoo. The training environment of Ultralytics is preserved in the model-zoo so that customers can leverage known tools. This guide provides instructions on how to train YOLO-based models with custom datasets for Mythic's Analog Matrix Processor (AMP). The steps to train a model are:

1. Train the FP32 model with a custom dataset. This is used as a starting point for Mythic's analog-aware training.
2. Convert the FP32 PyTorch model to ONNX. Mythic uses ONNX as a standard format to its training tools. This allows frameworks other than PyTorch to generate the initial FP32 model.
3. Convert the FP32 model for analog-aware retraining.
4. Perform analog-aware retraining.
5. Verify the accuracy of the analog-aware model using a software simulation of the Mythic AMP. This is essential in confirming that the model will provide good silicon accuracy.
6. Compile and deploy the mode to silicon to confirm final latency and accuracy.

For customers setting up the Mythic training flow for the first time, we recommend you follow the steps below with a standard COCO dataset (e.g., for Pose). Once you have confirmed that each step can be completed and the silicon accuracy matches expectations, switch to using a custom dataset.

2 Hardware Requirements

Model retraining is a compute-intensive process that requires a PC with sufficient performance to meet training demands.

Mythic recommends the following system specifications:

- Intel Core i7 CPU or better
- 64GB of RAM
- NVIDIA GPU with 48GB of VRAM
- Ubuntu 22.04 or newer

Lower performance systems may also work, but they will require tuning hyperparameters such as batch size to fit the model in memory.

For more information, please contact Mythic's Support Team.

3 Prerequisites

The following is required for analog-aware training:

1. A workstation with a GPU and CUDA installed. See the above for the recommended GPU configuration.
2. Mythic's full SDK has been installed. See the 'M.2 Accelerator Card User Guide' for details on how to install the SDK. If the training system does not contain an M1076C, installation of Mythic's driver can be skipped.

4 Ultralytics' `yolo` CLI

This document uses Ultralytics yolo CLI for tasks such as FP32 training or dataset download. We recommend that customers unfamiliar with this tool visit <https://docs.ultralytics.com> to review its full functionality. This tool uses the following syntax:

MYTHIC

```
yolo TASK MODE ARGS
```

Where TASK (optional) is one of ('detect', 'segment', 'classify', 'pose', 'obb', 'segpose')

MODE (required) is one of ('train', 'val', 'predict', 'export', 'track', 'benchmark')

ARGS (optional) are any number of custom 'arg=value' pairs like 'imgsz=320' that override defaults.

See all ARGS at <https://docs.ultralytics.com/usage/cfg> or with 'yolo cfg'

Care must be taken to ensure the TASK (object detection = `detect`, instance segmentation = `segment`, and pose detection = `pose`) uses the correct model type and data in the ARGS. For example, to train pose detection, the following syntax is used:

```
yolo train pose model=yolov8s-pose.yaml data=coco-pose.yaml
```

 The SDK includes a modified version of Ultralytics' Python package. Do not install the original Ultralytics package in the `mythic-model-zoo` Python environment since this will have unwanted side effects.

5 Task Configuration

1. Set the environment variable `MYTHIC_SDK_ROOT` to point to the SDK directory. For example,

```
export MYTHIC_SDK_ROOT=~/.mythic_sdk/v26.05.0/
```

2. Go to the model-zoo source code directory:

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo
```

3. Mythic's analog-aware training needs certain environment variables to be set depending on the task being trained. Scripts are provided to activate the `mythic-model-zoo` Python environment and the environment variables for the training task you want to work with.

- a. For Pose detection on M1076C:

```
./scripts/yolov8/pose.env
```

- b. For Pose detection on M2000:

```
./scripts/yolov8/pose-m2000.env
```

- c. For Object detection on M1076C:

```
./scripts/yolov8/detect.env
```

d. For Object detection on M2000:

```
./scripts/yolov8/detect-m2000.env
```

e. For Instance Segmentation on M1076C:

```
./scripts/yolov8/segment.env
```

f. For Instance Segmentation on M2000:

```
./scripts/yolov8/segment-m2000.env
```

The M1076C environment setup scripts will also set the environment variable `IPU_PCI_ADDRESS`. This variable defines which AMP to use. If there is only one AMP, the variable will be set correctly. If there are multiple AMPs, you must set the variable to the hexadecimal address of the AMP you want to use. To list available AMPs, you can use the command `lspci | fgrep accel`. This step can be ignored for systems that do not contain an AMP processor.

The Mythic model-zoo scripts use Hydra to handle configuration files and command line parameters. The examples in the following sections show some basic ways of overriding configuration parameters. For advanced usage, please refer to the Hydra documentation.

5.1 Dataset Setup

5.1.1 Using Standard Datasets

Ultralytics supports COCO datasets for training Pose, Segmentation, and Object Detection tasks. The following table summarizes the datasets and configuration files associated with each task:

Task	Full dataset	Ultralytics Config File
Object Detection	COCO	coco.yaml
Pose	COCO-Pose	coco-pose.yaml
Instance Segmentation	COCO	coco.yaml

These datasets can be automatically downloaded by running Ultralytics' CLI in validation mode. For example,

1. Download COCO by running validation on yolov8n object detection FP32 model

MYTHIC

```
yolo val data=coco.yaml model=yolov8n
```

2. Download COCO-pose by running validation on yolov8n pose FP32 model

```
yolo val data=coco-pose.yaml model=yolov8n-pose
```

These commands will download the respective dataset to `$MYTHIC_SDK_ROOT/mythic-model-zoo/datasets`.

5.1.2 Using a Custom Dataset

Mythic's SDK uses Ultralytics' training software and the Ultralytics' dataset format. The steps to set up a custom set-up are:

1. Add training and validation images to your workstation.
2. Create a dataset configuration file as described in Ultralytics documentation <https://docs.ultralytics.com/datasets/>. The dataset configuration file should be stored in `configs/yolov8/datasets` directory.

This directory already contains dataset configuration files (`mythic-*.yaml`) that Mythic uses for training networks on its internal GPU infrastructure. These files can be used as a reference for creating new dataset configuration files.

The following table defines the maximum number of classes that can be supported for each task:

Task	Limitation	Comment
Object Detection	<= 80 classes	If the dataset contains less than 80 classes, the model configuration file must be edited to match the number of classes in the dataset. See below for details.
Pose	17 keypoint positions	The dataset must contain 17 keypoint positions
Segmentation	<= 80 classes	If the dataset contains less than 80 classes, the model configuration file must be edited to match the number of classes in the dataset. See below for details.

The Ultralytics `yolo` <https://docs.ultralytics.com/usage/cli/> can be used to validate the dataset configuration file is structured correctly. For example, for Pose:

```
yolo val data=configs/yolov8/datasets/your-dataset.yaml model=yolov8n-pose
```

To use the custom dataset in Mythic's training tools, the environment variable `DATASET` must be set to the path of your dataset configuration file. For example,


```
export DATASET=configs/yolov8/datasets/your-dataset.yaml
```

6 Model Configuration Files

Mythic's SDK ships with structurally modified versions of the original Ultralytics models. The modifications remove unsupported nodes (like swapping SiLU to ReLU). The modified models are located in

`$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/models`.

Ultralytics support five model sizes by scaling the model based on a size parameter encoded in the model name (nano='n', small='s', medium='m', large='l' and extra large='x'). Mythic uses the 'small' model size since this offers a good compromise of accuracy versus model size. The following table shows yaml file for each task and the required syntax to get the small model.

Task	Model YAML file	Syntax for small model
Object Detection (detect)	yolov5-p6.yaml	yolov5s-p6.yaml
Pose Detection (pose)	yolov8-pose_relu.yaml	yolov8s-pose_relu.yaml
Instance Segmentation (segment)	yolov8-seg_relu.yaml	yolov8s-seg_relu.yaml

 Failure to include the model size in the model name will cause Ultralytics to default to the nano model. This will result in poor model accuracy.

 For customers creating 'new' model YAML files with a smaller number of classes, do not add the model size to the YAML file name. This must be parsed at runtime.

6.1 Reducing number of classes in a model configuration file

For objection detection and segmentation, the total number of classes supported by the model must be edited to match the dataset. The following steps describe changing the segmentation model to support four classes.

1. Edit the new model base configuration file (e.g., `vi yolov8-seg_relu.yaml`). Modify the `nc` parameter to four, as shown in the code snippet below.

```
Ultralytics YOLO 🚀, AGPL-3.0 license
# YOLOv8-seg instance segmentation model. For Usage examples see https://
docs.ultralytics.com/tasks/segment

# Parameters
nc: 4 # number of classes
```

MYTHIC

```
scales: # model compound scaling constants, i.e. 'model=yolov8n-seg.yaml' will
call yolov8-seg.yaml with scale 'n'
# [depth, width, max_channels]
n: [0.33, 0.25, 1024]
s: [0.33, 0.50, 1024]
m: [0.67, 0.75, 768]
l: [1.00, 1.00, 512]
x: [1.00, 1.25, 512]
activation: nn.ReLU()
```

7 Conversion and Retraining Steps

Each of the following steps generates an output file. Please confirm that a valid output has been generated before proceeding to the next step.

7.1 Generate a Trained Floating Point Model

Using the modified version of the Ultralytics Python package provided in the SDK and the existing configuration files under `configs/yolov8/models/` train a floating point model. You can follow the instructions in the <https://docs.ultralytics.com/usage/cli/> to tune hyperparameters. Here are examples for Object Detection, Pose, and Segmentation.

1. Objection detection

```
yolo detect train model='configs/yolov8/models/yolov5s-p6.yaml' \
data='configs/yolov8/datasets/your_dataset.yaml' epochs=... imgsz=...
```

2. Pose

```
yolo pose train model='configs/yolov8/models/yolov8s-pose_relu.yaml' \
data='configs/yolov8/datasets/your_dataset.yaml' epochs=... imgsz=...
```

3. Segmentation

```
yolo segment train model='configs/yolov8/models/yolov8s-seg_relu.yaml' \
data='configs/yolov8/datasets/your_dataset.yaml' epochs=... imgsz=...
```

When training starts, the parameters being used for training will be output. Verify that critical parameters like dataset, epochs, and image size are as expected. For example, the below shows the output for one epoch of segmentation training with COCO.

```
engine/trainer: task=segment, mode=train, model=configs/yolov8/models/yolov8s-
seg_relu.yaml, data=coco.yaml, epochs=1, time=None, patience=50, batch=16, imgsz=640,
save=True, save_period=-1, cache=False, device=None, workers=8, project=None,
name=train9, exist_ok=False, pretrained=True, optimizer=auto, verbose=True, seed=0,
```


MYTHIC

```
deterministic=True, single_cls=False, rect=False, cos_lr=False, close_mosaic=10,
resume=False, amp=True, fraction=1.0, profile=False, freeze=None, multi_scale=False,
overlap_mask=True, mask_ratio=4, dropout=0.0, val=True, split=val, save_json=False,
save_hybrid=False, conf=None, iou=0.7, max_det=300, half=False, dnn=False, plots=True,
source=None, vid_stride=1, stream_buffer=False, visualize=False, augment=False,
agnostic_nms=False, classes=None, retina_masks=False, embed=None, show=False,
agnostic_nms=False, save_txt=False, save_conf=False, save_crop=False, show_labels=True,
show_conf=True, show_boxes=True, line_width=None, format=torchscript, keras=False,
optimize=False, int8=False, dynamic=False, simplify=False, opset=None, workspace=4,
nms=False, lr0=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0,
warmup_momentum=0.8, warmup_bias_lr=0.1, box=7.5, cls=0.5, dfl=1.5, pose=12.0, kobj=1.0,
label_smoothing=0.0, nbs=64, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0,
translate=0.1, scale=0.5, shear=0.0, perspective=0.0, flipud=0.0, fliplr=0.5,
mosaic=1.0, mixup=0.0, copy_paste=0.0, auto_augment=randaugment, erasing=0.4,
crop_fraction=1.0, cfg=None, tracker=botsort.yaml, save_dir=runs/segment/train9
```

The final output is a file called `best.pt`. By default, Ultralytics puts this file in a subdirectory called `runs/`
`<task>/<training#>`. For pose detection, the file is `runs/pose/training`.

7.2 Export the Trained Floating Point Model to ONNX

The trained floating point model should be exported to ONNX using the yolo CLI <https://docs.ultralytics.com/modes/export/>. It is required that you use `opset=11`

```
yolo export model=/your/path/to/the/best.pt format=onnx opset=11
```

- **Output:** The trained ONNX model `/your/path/to/the/best.onnx`

7.3 Convert the Floating Point model to Structural

A “structural” model is an intermediate model in the conversion process from floating point to retraining. To obtain the structural model run:

```
python3 scripts/common/convert_model.py steps=to_structural fp_model=/your/path/to/the/
best.onnx
```

Output: The converted model will be saved under `data/structural.onnx`

7.4 Convert the Structural Model to Retraining

Conversion from structural to retraining is done with the `to_training` step:

```
python3 scripts/common/convert_model.py steps=to_training
```

MYTHIC

When conversion starts, verify that parameters such as a dataset are correct. For example, the output below is for converting the segmentation model using coco.

```
engine/trainer: task=segment, mode=train, model=None, data=coco-seg.yaml, epochs=100,
time=None, patience=50, batch=1, imgsz=320, save=True, save_period=-1, cache=False,
device=None, workers=8, project=None, name=train10, exist_ok=False, pretrained=True,
optimizer=auto, verbose=True, seed=0, deterministic=True, single_cls=False, rect=False,
cos_lr=False, close_mosaic=10, resume=False, amp=True, fraction=1.0, profile=False,
freeze=None, multi_scale=False, overlap_mask=True, mask_ratio=4, dropout=0.0, val=True,
split=val, save_json=False, save_hybrid=False, conf=None, iou=0.7, max_det=300,
half=False, dnn=False, plots=True, source=None, vid_stride=1, stream_buffer=False,
visualize=False, augment=False, agnostic_nms=False, classes=None, retina_masks=False,
embed=None, show=False, save_frames=False, save_txt=False, save_conf=False,
save_crop=False, show_labels=True, show_conf=True, show_boxes=True, line_width=None,
format=torchscript, keras=False, optimize=False, int8=False, dynamic=False,
simplify=False, opset=None, workspace=4, nms=False, lr0=0.01, lrf=0.01, momentum=0.937,
weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1,
box=7.5, cls=0.5, dfl=1.5, pose=12.0, kobj=1.0, label_smoothing=0.0, nbs=64,
hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0, translate=0.1, scale=0.5, shear=0.0,
perspective=0.0, flipud=0.0, flip_lr=0.5, mosaic=1.0, mixup=0.0, copy_paste=0.0,
auto_augment=randaugument, erasing=0.4, crop_fraction=1.0, cfg=None,
tracker=botsort.yaml, save_dir=runs/segment/train10
```

Output: The retraining ONNX `data/mythic.onnx`

7.5 Retrain

The `data/mythic.onnx` can be retrained in the `train` step. This is followed by the `update_onnx_weights` to save the trained network as an ONNX file in the `data` directory. The value of the `device` parameter below is a list of GPUs to use.

```
python3 scripts/common/convert_model.py steps=[train,update_onnx_weights] \
++train.wandb.mode=disabled \
device=0,1,2,3
```

As per the previous step, check that training parameters, such as the dataset, are expected the expected values once training starts.

Output: The retrained ONNX `data/trained.onnx`

7.5.1 Hyper-parameters

Hyper-parameters (batch size, learning rate, ... etc) can be specified in a configuration file in `configs/yolov8/training/training_config/`. The default file for each task is shown in the table below:

Task	Training configuration file
Object Detection	yolov5s6u-retrain.yaml
Pose	yolov8s-pose.yaml
Segmentation	yolov8s-seg.yaml

To change parameter values, edit this file or create a new one in the same directory and put its name into the `TRAINING_SETUP` environment variable. These training parameters are passed to the Ultralytics training logic. They are documented on the Ultralytics website <https://docs.ultralytics.com/usage/cfg/#train>.

7.5.2 Retraining Parameters

Retraining takes the following parameters:

Configuration parameter or group	Description	Default value
src	A model to train. A value can be a local file name or an URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .	<code>\${mythic_model}</code> , which defaults to <code>data/mythic.onnx</code> .
ultralytics_config (group)	Training parameters to be passed to the Ultralytics training logic. See https://docs.ultralytics.com/usage/cfg/#train	<code>\${training_config}</code> , which defaults to <code>\${oc.env:TRAINING_SETUP,yolov8s-pose}</code> , which loads configuration from <code>configs/yolov8/training/ultralytics_training_config/yolov8s-pose.yaml</code> .
run_dir	A directory to save training checkpoints and results.	If not specified, a uniquely named subdirectory of <code>data/training/</code> is used. Created sub-directories are sequentially numbered. The subdirectory with the larger number contains the results of the last run.

Configuration parameter or group	Description	Default value
run_name	A name of this run. It's used as a W&B run name if W&B is installed and configured.	Defaults to the name of the run directory (run_dir).
wandb (group)	wandb.init parameters	Two keys are set by default: <code>project</code> - default to <code>yolov8-pose</code> and <code>name</code> - defaults to the value of parameter <code>run_name</code> .

To specify a step parameter value on the command line use `train.parameter=value` syntax.

7.6 Change Model Input Resolution

This step is only applicable to 1080p M2000 models. It changes the input resolution from 1920x1920, which is used for training, to 1088x1920, which is used for inference.

```
mv data/trained.onnx data/trained-1920x1920.onnx
python scripts/common/convert_model.py steps=change_input_resolution \
  change_input_resolution.src=data/trained-1920x1920.onnx \
  change_input_resolution.dest=data/trained.onnx \
  change_input_resolution.imgsz=[1088,1920]
```

7.7 Optimize per-channel ADC ranges (optional)

This step currently does not improve accuracy.

```
python3 scripts/common/convert_model.py steps=optimize_ifsr
```

- **Output:** An optimized retrained ONNX `data/optimized_ifsr.onnx`

This ONNX can be used in the accuracy evaluation steps by using `trained_model=data/optimized_ifsr.onnx`.

7.8 Convert to Evaluation Model

Convert the retrained `data/trained.onnx` to an evaluation model:

```
python scripts/common/convert_model.py steps=to_acm
```

MYTHIC

- **Output:** The evaluation model `data/acm.onnx`

7.9 Compute Accuracy Metrics

7.9.1 M2000

Compute detailed metrics on the evaluation model with the command:

```
python3 scripts/common/convert_model.py steps=eval_trained
```

- **Output:** The accuracy metrics `data/metrics.json`.

Compute Monte Carlo-based 100PPM estimates:

```
python3 scripts/common/convert_model.py steps=mc_eval_trained  
mc_eval_trained.num_samples=100
```

- **Output:** One file per sample in `data/collected_metrics/`

Print the 100PPM estimates after collecting samples:

```
python3 scripts/common/convert_model.py steps=mc_summarize_metrics
```

- **Output:** The accuracy 100PPM metrics `data/collected_metrics/metrics_summary.json`

7.9.2 M1076C

Compute detailed metrics on the evaluation model with the command:

```
python3 scripts/common/convert_model.py  
steps=[eval_fp,eval_digital,eval_signoff_v0p4,eval_signoff_v0p5]
```

- **Output:** The accuracy metrics `data/metrics.json`.

The `eval_...` steps compute accuracy metrics using different Mythic hardware models. `eval_fp` uses a floating point model that does not apply digital quantization effects or analog noise. `eval_digital` uses a model that applies digital quantization effects but no analog noise. `eval_signoff_v0p4` and `eval_signoff_v0p5` apply digital quantization effects and an analog noise model.

- The `eval_digital` may be ~2% lower than the `eval_fp`. If it is significantly lower, silicon accuracy will be low. It is recommended that you train with a smaller learning rate.
- The `eval_signoff_v0p4` and `eval_signoff_v0p5` may be ~10% lower than the `eval_fp`. If they are significantly lower, silicon accuracy will be low. One possible cause for this is a small dataset.

7.10 Generate Artifact

Create an **artifact**, namely a set of files that includes the best model weights. The compiler will use the artifact to generate the firmware that is programmed into the Mythic AMP™ for inference.

To generate an artifact from the existing `data/acm.onnx` :

```
python3 scripts/common/convert_model.py steps=create_artifact \
  artifact=data/compiler_ready_artifact.tar.gz
```

- **Output:** The training artifact `data/compiler_ready_artifact.tar.gz`

Alternatively, you can override the defaults with the following parameters:

Configuration parameter or group	Description	Default value
src	An ONNX evaluation model to generate an artifact from.	<code>\${acm_model}</code> , which defaults to <code>data/acm.onnx</code> .
dest	An ONNX file to save the converted model to. A value can be a local file name or a URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .	<code>\${env_setup.artifact_path}</code> , which defaults to <code>\${artifact}</code> , which default to <code>data/compiler_ready_artifact.tar.gz</code>

To specify a parameter value on the command line, use `create_artifact.parameter=value` syntax, e.g., `create_artifact.dest=gs://some_location/my_artifact.tar.gz` .

7.11 Firmware Compilation

Once the artifact has been generated, the output can be run with the compiler to create a binary that can be loaded for inference. To generate a firmware binary with the compiler, use the `mythic-compiler` script with a compiler configuration file (located in `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler`) for the model you want to use. The table below enumerates the compiler configuration files supported in the SDK:

Task	Input Resolution	Compiler Configuration file
Instance segmentation	640x640	yolov8s_seg_640x640x3_full_compile.yaml

MYTHIC

Task	Input Resolution	Compiler Configuration file
Object detection	1408x1408	yolov5s6u_1408x1408x3.yaml
Pose detection	640x640	yolov8s_pose_640x640x3.yaml
Pose detection	736x1280	yolov8s_pose_736x1280x3.yaml

The following examples demonstrate how to compile a pose-detection artifact trained at 640x640 resolution run and pose-detection artifact at 1280x720.

1. Pose trained at 640x640:

```
mythic-compiler --compiler-config configs/yolov8/compiler/
yolov8s_pose_640x640x3.yaml \
--input-artifact TRAINING_ARTIFACT.tar.gz --output-artifact FIRMWARE.tar.gz
```

2. Pose trained at 1280x720:

3.

```
mythic-compiler --compiler-config configs/yolov8/compiler/
yolov8s_pose_736x1280x3.yaml \
--input-artifact TRAINING_ARTIFACT.tar.gz --output-artifact FIRMWARE.tar.gz
```

Output: The firmware `FIRMWARE.tar.gz`

`mythic-compiler` takes the following parameters:

Parameter	Description
<code>--compiler-config</code>	A compiler recipe to use. Currently for the Pose model Mythic provides recipes for 640x640 and HxW=1280x720 image resolutions: <code>configs/yolov8/compiler/yolov8s_pose_640x640x3.yaml</code> . <code>configs/yolov8/compiler/yolov8s_pose_736x1280x3.yaml</code> .
<code>--input-artifact</code>	A name of a training artifact generated by the steps described above.

Parameter	Description
<code>--output-artifact</code>	A name of a firmware artifact to be generated. A value can be a local file name or an URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .

7.12 Benchmark AMP Inference (M1076C only)

To run a compiled model on an AMP, use the script `scripts/common/benchmark_amp_inference.py`. This script can be used to execute Ultralytics' validation and predict modes. Before running it, make sure the required environment variables are set as described in the previous sections. The inference script uses an AMP specified in the environment variable `IPU_PCI_ADDRESS`. The pre-trained models in `$MYTHIC_SDK_ROOT/models` can be used to test `benchmark_inference_amp.py` with standard COCO datasets without training a model from scratch.

The `benchmark_amp_inference.py` script takes the following parameters:

Configuration parameter or group	Description	Default value
<code>src</code>	A firmware artifact to benchmark. A value can be a local file name or a URL of a remote file supported by https://cloudpathlib.drivendata.org/stable/ .	
<code>dataset</code>	Only used in validation mode. A dataset to use for measuring accuracy metrics. It should be a path to a dataset configuration file.	The value of environment variable <code>DATASET</code> , or, if not set, <code>coco-pose.yaml</code> .
<code>eval_silicon.log_file</code>	A file json file to save results to.	A timestamped file <code>data/benchmark_amp_inference_{\$now:%Y-%m-%dT%H%M%S}.json</code>
<code>input</code>	Only used in prediction mode. Points to a video file, image file or directory of images	

 Benchmark_mark_inference.py uses the model configuration file (e.g., yolov8-seg_rel.yaml) to determine the number of classes the model supports and also uses the dataset configuration file (e.g., custom_data.yaml) to determine where the validation dataset is located. For customers with separate workstations for training and inference, these configuration files must be available on the inference system.

Validation Mode

Validation mode is used to measure accuracy metrics for a model. The input to this script is the compiled firmware image described in the previous step. For example,

```
python3 scripts/common/benchmark_amp_inference.py src=/PATH/TO/FIRMWARE.tar.gz
```

Prediction Mode

Prediction mode can be used to process video input files or directories of images. It renders the inference result over the input content in files located in the runs `directory` . For example,

```
python3 scripts/common/benchmark_amp_inference.py -cn amp_predict src=/PATH/TO/FIRMWARE.tar.gz input=test_video.mp4
```